

SecurePay: Cryptographic Security Audit

Individual Digital Investigation Assignment

Course: Information Security Report: 60 marks Individual evaluation: 40 marks

Expected effort: approximately 5–6 hours. The same case and evidence package is used for every student.

Incident brief

SecurePay, a fictional digital-payment company, advertises DES, AES, hashes, MACs, RSA, Diffie–Hellman, signatures, and certificates as proof that its platform is secure. A major incident shows that algorithm names alone are not a security architecture. You are the independent investigator. Determine what failed, support each finding with relevant evidence, and propose a defensible SecurePay 2.0 design.

Your investigation should normally follow: **action/experiment** → **observation** → **result** → **evidence** → **security explanation** → **recommended fix**. Generic theory without engagement with the supplied evidence earns limited credit.

Submission and tools

Submit **one word file investigation report only**. Do not submit source code, Python files, scripts, notebooks, ZIP archives, program source, or tool-configuration files. You may use Python, standard cryptographic libraries, OpenSSL, CyberChef, calculators, or another appropriate tool. You are not required to implement cryptographic primitives.

Show intermediate manual working where explicitly requested for the toy CBC-MAC, textbook RSA, and educational Diffie–Hellman tasks. For real AES, SHA-256, HMAC, signatures, and certificates, tool output is appropriate. Source-code screenshots are not required.

Evidence standard. Screenshots alone are not solutions. Use cropped, relevant, readable output; recovered text; before/after values; compact tables; or calculation steps. A result without explanation, or an explanation without a supporting result/evidence item, receives limited credit. Avoid full-screen or repetitive screenshots. Conciseness is valued. A well-structured report will usually be about 7–10 pages excluding its cover, but there is no page minimum.

Individual evaluation. Any concept or investigation from this assignment may be examined individually. You must understand and defend every part of your submitted report. The later evaluation is worth 40 marks.

How to use the evidence package

The supplied files are **investigation inputs**, not files to submit. A binary file is not expected to look meaningful when opened in a text editor; it must be supplied unchanged to an appropriate cryptographic tool. Read the task and the manifest in that evidence folder before using its files, preserve the originals, and perform modifications only on copies.

Type	What it contains	How students use it
.txt, .log	Human-readable messages, captured values, algorithms, timelines, or audit records.	Open and read first. Copy exact values into a calculator/tool where needed. Follow stated encoding and newline rules.
.pem	A standard text-armoured RSA key, X.509 certificate, or CRL.	Give the complete file to OpenSSL, a certificate viewer, CyberChef, or a cryptographic library. A public certificate/key is inspected or used for verification; the Part 3 private key is a disposable training key used only to unwrap that exercise's session key.
.bin	Raw ciphertext or a wrapped session key.	Do not edit as ordinary text. Supply the file as binary input to the operation named in its manifest. In Part 1B, modify only a working copy at the required byte positions.
.sig	Raw digital-signature bytes.	Use with the stated message, public key, and signature algorithm in Part 5. It is evidence to verify, not text to read.
.b64	A Base64 text representation of the same signature.	This is an alternative to the raw .sig for tools that accept Base64; it is not a second signature task.
.der	A standard binary OCSP response.	Inspect with an OCSP/certificate-capable tool. The matching .txt file is a readable copy for students whose tool cannot parse DER.

Evidence roadmap.

Part	Files	Purpose
1	ecb_capture.txt, cbc_case.txt, cbc_original.bin, ctr_capture.txt, ctr_known_plaintext.txt	Compare ECB blocks; modify/decrypt a copy of the CBC ciphertext; recover the second CTR plaintext from two captures and known plaintext.
2	hash_packet.txt, toy_cbc_mac_case.txt, hmac_case.txt	Verify and change an ordinary hash; perform the manual toy CBC-MAC chain; verify and change HMAC input.
3	rsa_math_case.txt; then envelope_manifest.txt, the RSA key files, wrapped_session_key.bin, and encrypted_report.bin	Complete textbook RSA working; then unwrap the training AES key and decrypt/authenticate the report exactly as described by the manifest.
4	dh_exchange.txt, mitm_capture.txt	Source values for the required manual normal-DH and MITM calculations.
5	signature_manifest.txt, message/signature/public-key files; certificate PEM files; certificate_incident.txt; enrollment_audit.log	Verify the signed transcript and a modified copy; inspect the certificate path and key binding; diagnose the RA/CA process failure.
6	revocation_timeline.txt, CRL PEM/summary, and OCSP DER/text response	Correlate status evidence. The summary/text forms are accessible alternatives to the standard binary forms, not additional separate problems.

If a chosen tool cannot open a standard binary format, use the supplied human-readable counterpart where one is provided, or choose another permitted tool. The report needs the relevant result and evidence—not the tool’s source code.

Suggested report template

For each numbered investigation, use only the headings that help: *Finding; Action/Experiment; Result; Evidence; Calculation/Working; Security Explanation; Recommended Fix*. End with the required architecture diagram and security-property table.

1 Part 1 – Block-cipher incident (12 marks; 60–75 minutes)**1A. ECB record leakage (4 marks)**

Inspect `Evidence/Part1_Block_Ciphers/ecb_capture.txt`. Identify every repeated ciphertext block that is significant, relate it to the corresponding structured field, and explain what an observer can infer without recovering the AES key. Evaluate all three statements: (i) “AES has been broken,” (ii) replacing AES-ECB with DES-ECB fixes the leak, and (iii) DES creates an additional key-strength problem. Recommend a production-appropriate correction.

SecurePay engineer claim: “Because every record is encrypted with AES, transaction structure cannot leak.”

Report: state the finding, action/experiment, result, concise evidence, security explanation, and recommended fix where applicable.

1B. CBC modification (4 marks)

Use `cbc_case.txt` and a working copy of `cbc_original.bin`. Starting from

$$P_i = D_K(C_i) \oplus C_{i-1},$$

determine a 16-byte XOR mask and alter the appropriate ciphertext block so that the second decrypted block becomes exactly `ROLE=ROOT;OK=YES`. Decrypt your modified ciphertext. Report the mask, the modified ciphertext block, the resulting three plaintext blocks (hex is acceptable for corrupted data), and which effects were controllable. Explain why encryption alone did not protect authorization data and recommend a fix.

SecurePay engineer claim: “CBC encryption rejects any attacker change because the attacker does not know the key.”

Report: state the finding, action/experiment, result, concise evidence, security explanation, and recommended fix where applicable.

1C. CTR nonce reuse (4 marks)

Use `ctr_capture.txt` and `ctr_known_plaintext.txt`. Recover the complete second plaintext using the captured values and the nonce-reuse relationship $C_1 \oplus C_2 = P_1 \oplus P_2$. Include the recovered transaction and compact evidence of your XOR process. State precisely which reuse condition made recovery possible, whether changing only the AES key size would help, and how SecurePay should manage CTR counters/nonces and integrity.

SecurePay engineer claim: “CTR is a stream mode, so reusing a counter value is harmless if AES-256 is used.”

Report: state the finding, action/experiment, result, concise evidence, security explanation, and recommended fix where applicable.

2 Part 2 – Hashes and message authentication (9 marks; 50–60 minutes)

2A. Hash appended to a message (2 marks)

Inspect `Evidence/Part2_Message_Authentication/hash_packet.txt`. Verify its digest. Then make the controlled change `amount=125000` to `amount=925000`, recompute SHA-256 under the stated encoding rule, and show the forged message-plus-digest pair. Explain why an unkeyed hash sent through the same attacker-controlled channel is not message authentication, while noting when an unkeyed hash can still be a useful integrity check.

2B. Educational CBC-MAC (3 marks)

Using `toy_cbc_mac_case.txt`, manually complete a table with columns i , M_i , $X_i = M_i \oplus T_{i-1}$, and $T_i = E(X_i)$ for both the original and captured candidate. Decide whether the candidate's attached tag is authentic. Show every intermediate byte in hex. State why this tiny construction is for learning only and why message-length handling matters in real CBC-MAC use.

2C. HMAC experiment (4 marks)

Use `hmac_case.txt`. Recompute and verify the captured HMAC-SHA-256. Apply the required one-digit amount change and compute the new HMAC. Report both values and the verification decisions. Using

$$\text{HMAC}_K(M) = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel M)),$$

explain why the key matters, how HMAC differs from an ordinary hash, which properties it does and does not provide, and why a standardized construction is preferable to inventing $\text{Hash}(K \parallel M)$.

3 Part 3 – RSA and a digital envelope (8 marks; 45–55 minutes)

3A. Textbook RSA mathematics (3 marks)

From `Evidence/Part3_RSA_Digital_Envelope/rsa_math_case.txt`, manually calculate $n = pq$, $\phi(n) = (p - 1)(q - 1)$, the private exponent d satisfying $ed \equiv 1 \pmod{\phi(n)}$, the ciphertext $c = m^e \bmod n$, and the recovered $m' = c^d \bmod n$. Show compact modular working and explain why these deliberately small, unpadded values are not a production recipe.

3B. Recover a digital envelope (5 marks)

Follow `envelope_manifest.txt` using the four files in the same evidence directory. Unwrap the AES session key with the supplied training RSA private key, then authenticate and decrypt the AES-256-GCM report with the supplied nonce and additional authenticated data. Report the recovered session-key hex, report approval code, recovered-report SHA-256, and concise successful-decryption evidence. Explain why SecurePay uses a hybrid envelope rather than encrypting a financial report directly with RSA, and distinguish this OAEP exercise from raw textbook RSA.

4 Part 4 – Diffie–Hellman and MITM (10 marks; 50–60 minutes)

4A. Normal exchange (4 marks)

Using `Evidence/Part4_DH_MITM/dh_exchange.txt`, manually recompute $A = g^a \bmod p$, $B = g^b \bmod p$, and both forms $K = B^a \bmod p$ and $K = A^b \bmod p$. Show enough modular working to be checked and confirm equality.

4B. Reconstruct the interception (6 marks)

Using `mitm_capture.txt`, calculate the Alice–Mallory secret two equivalent ways and the Mallory–Bank secret two equivalent ways. Draw the replaced-public-value flow. Explain, step by step, how two different secrets allow Mallory to decrypt, inspect or modify, and re-encrypt traffic. State the precise missing property in bare Diffie–Hellman and recommend how SecurePay should authenticate the exchange.

SecurePay engineer claim: “Diffie–Hellman proves that the shared secret is with the bank.”

Report: state the finding, action/experiment, result, concise evidence, security explanation, and recommended fix where applicable.

5 Part 5 – Digital signatures and PKI (10 marks; 45–55 minutes)

5A. Signature experiment (4 marks)

Follow `Evidence/Part5_Signatures_PKI/signature_manifest.txt`. Verify the supplied signature over the original `dh_parameters.txt`. In a copy, make the specified `payment-channel` to `payout-channel` change and verify again with the unchanged signature. Record the two verification outcomes and concise evidence. Explain what a valid signature can establish in context (integrity, origin authentication, and possible non-repudiation) and why it provides neither confidentiality nor, by itself, real-world identity.

5B. Certificate path and key binding (4 marks)

Treat `trusted_root_ca.pem` as the configured trust anchor and inspect the server and intermediate certificates using `certificate_incident.txt`. Report the server Subject, Issuer, serial, DNS identity, validity interval, public-key type/size, and SHA-256 fingerprint. Establish the issuer path and signature relationships, identify the trust anchor, and compare the public key in `securepay_signing_public.pem` with the server certificate's key. Make a trust decision for the stated 09:15 connection, explicitly excluding the later revocation evidence until Part 6.

SecurePay engineer claim: “A verified signature means any public key used must belong to SecurePay.”

5C. Enrollment control failure (2 marks)

Analyze `enrollment_audit.log`. Identify what the Registration Authority should have validated, what the Certificate Authority did, the primary process failure, and one CA-side control that could have limited the damage. Do not merely define CA and RA.

6 Part 6 – Revocation and SecurePay 2.0 (11 marks; 45–60 minutes)

6A. CRL/OCSP decision (4 marks)

Correlate `revocation_timeline.txt`, `cached_crl.pem` (or `cached_crl_summary.txt`), and `ocsp_response.der` (or `ocsp_response.txt`). At 14:00, determine what a client relying only on cached CRL 417 would observe and whether the server certificate should actually be accepted. Explain the timing gap. Compare CRL and OCSP in terms of freshness, list size/bandwidth, responder availability, privacy, caching, and failure policy; do not claim one is universally better.

6B. SecurePay 2.0 architecture (7 marks)

Produce one clear architecture/protocol diagram and a compact security-property table. Your redesign must address:

- bulk transaction confidentiality with AES in a safe construction and explicit unique nonce/IV handling;
- transaction integrity/authentication (AEAD or encrypt-then-MAC with HMAC, as justified);
- Diffie–Hellman key establishment with authenticated transcript/key shares;
- digital signatures and certificates, including chain, hostname, validity, and revocation checks;

- CA/RA responsibilities and key-compromise response;
- RSA-OAEP/AES digital envelopes where offline report delivery is genuinely needed; and
- rejection of DES and unsafe ECB or nonce reuse.

For each mechanism, label which property it supplies: confidentiality, integrity, endpoint authentication, identity binding, key establishment/distribution, non-repudiation (where appropriate), or certificate-status information. State at least two implementation/operational requirements that algorithms alone do not solve.

Mark summary

Part	1	2	3	4	5	6	Total
Marks	12	9	8	10	10	11	60

Reminder: submit one word file report only. Do not submit code or the evidence package.